

Лабораторна робота №2. Реалізація смарт-контракту або анонімної криптовалюти.

Виконав: ФБ-51мн Мелік-Казарян Сімон Артурович

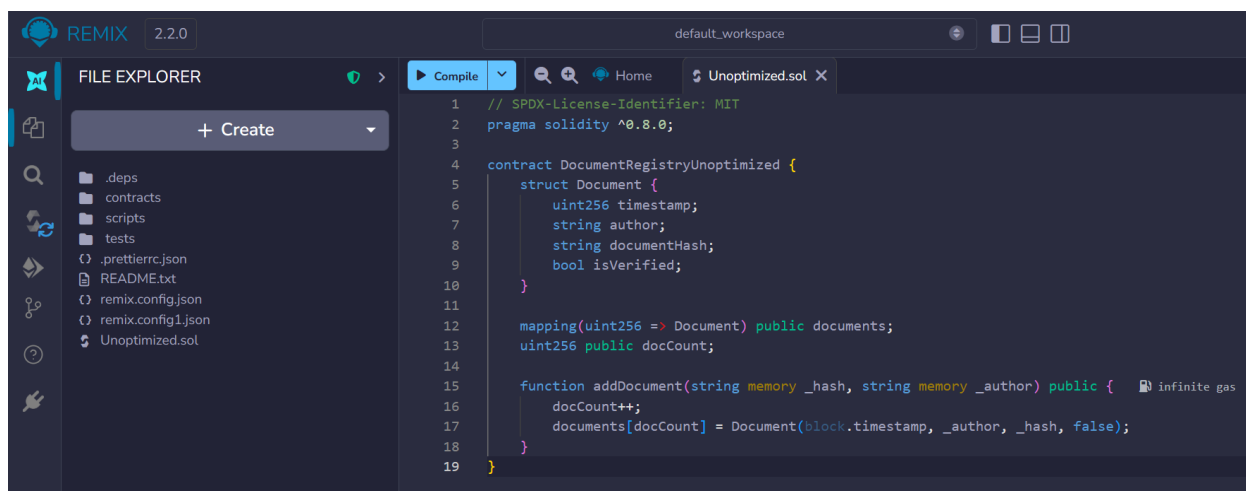
Мета роботи

Отримання навичок роботи із смарт-контрактами або анонімними криптовалютами

Хід роботи

Для розробки, оптимізації та розгортання смарт-контракту використаємо хмарне інтегроване середовище розробки Remix IDE. Воно містить вбудований компілятор мови Solidity та віртуальну машину Remix VM для безпечного тестування коду.

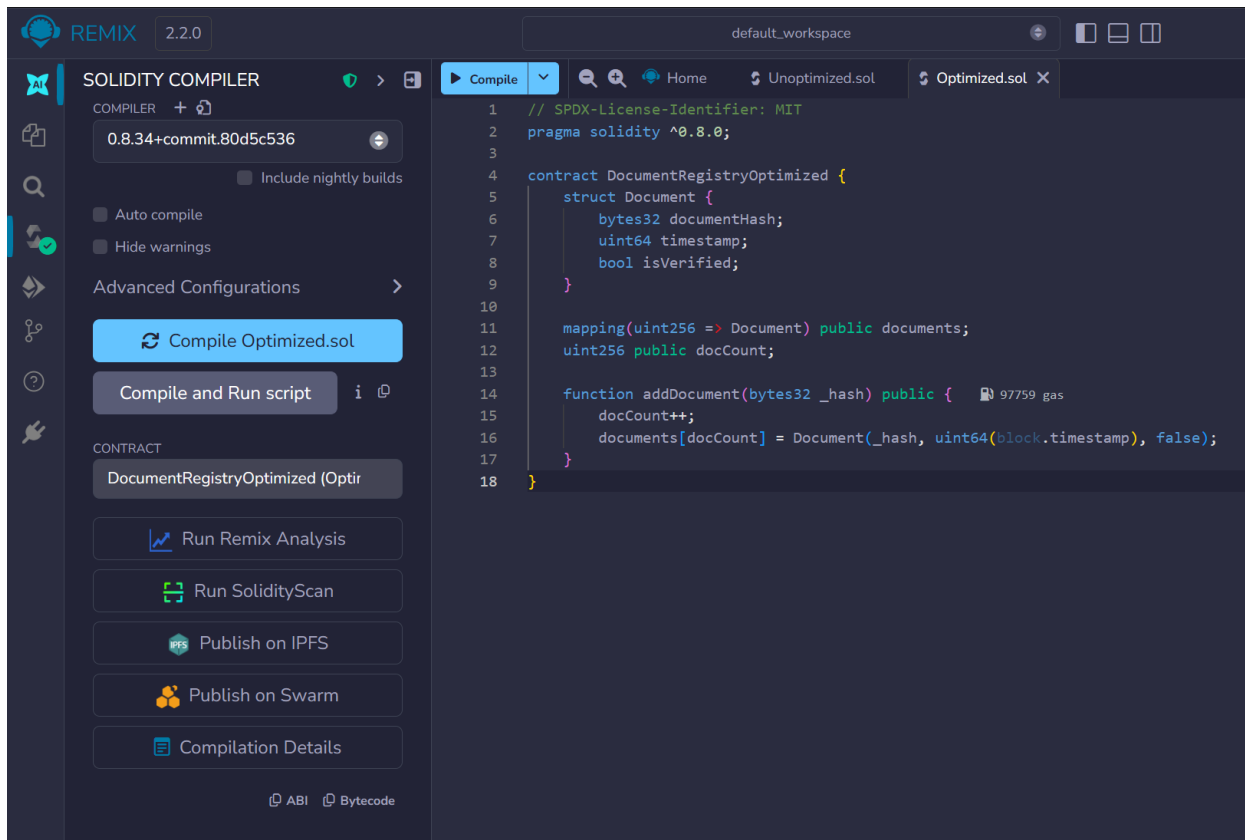
Напишемо смарт-контракт DocumentRegistry, який виконує роль децентралізованого реєстру для збереження хешів документів. Це дозволяє довести існування певного документа в певний момент часу. Базова, неоптимізована версія коду виглядає так:



```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract DocumentRegistryUnoptimized {
5     struct Document {
6         uint256 timestamp;
7         string author;
8         string documentHash;
9         bool isVerified;
10    }
11
12    mapping(uint256 => Document) public documents;
13    uint256 public docCount;
14
15    function addDocument(string memory _hash, string memory _author) public {
16        docCount++;
17        documents[docCount] = Document(block.timestamp, _author, _hash, false);
18    }
19 }
```

Будь-яка операція запису в блокчейн Ethereum коштує грошей (витрачається Газ). В оригінальному коді використання типу string для зберігання хешу є неефективним з точки зору обчислювальних ресурсів. Також порядок змінних у структурі можна запакувати, щоб вони займали менше слотів пам'яті (кожен слот - це 32 байти).

Створимо оптимізовану версію. Змінимо тип `string` `documentHash` на фіксований `bytes32`, а булеву змінну `isVerified` перемістимо так, щоб компілятор міг стиснути дані в один слот.



Скомпілюємо обидва контракти у вкладці Solidity Compiler та перейдемо до вкладки Deploy & Run Transactions. Розгорнемо їх у тестовій мережі Remix VM.

Після розгортання виконаємо функцію `addDocument` в обох контрактах і зафіксуємо витрати газу в логах консолі:

1. **Неоптимізований контракт:** запис документа (хеш типу `string` та автор) обійшовся у 94 015 gas (загальна вартість транзакції - 116 075 gas). Висока вартість зумовлена динамічним виділенням пам'яті під рядки..
2. **Оптимізований контракт:** запис аналогічного документа з використанням фіксованого типу `bytes32` обійшовся лише у 67 559 gas (загальна вартість транзакції - 89 135 gas).

transaction cost	116075 gas
execution cost	94015 gas
output	0x
decoded input	{ "string_hash": "test_hash", "string_author": "Author" }
decoded output	{}
logs	[]
raw logs	[]


[vm] from: 0x5B3...eddC4 to: DocumentRegistryOptimized.addDocument(bytes32) 0x358...D5eE3 value: 0 wei data: 0x582

status	1 Transaction mined and execution completed
transaction hash	0xd63352f18d99a9210174a5d3e89a92549688f0c2dae64da8f5618f7ff2380ebe
block hash	0xf27a542161965d4ab4a32b86030e45eb4119c5bb691446704a49c885d43315d7
block number	9
from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	DocumentRegistryOptimized.addDocument(bytes32) 0x358AA13c52544ECCEf6B0ADD0f801012ADAD5eE3
transaction cost	89135 gas
execution cost	67559 gas
output	0x
decoded input	{ "bytes32_hash": "0x1234567890123456789012345678901234567890123456789012345678901234" }

Висновок:

У ході роботи успішно спроектовано та розгорнуто власний смарт-контракт у середовищі Ethereum. Практично продемонстровано методи підвищення ефективності коду: заміна динамічних типів даних на фіксовані (string на bytes32) та оптимізація структури зберігання дозволили зменшити чисті витрати газу на виконання транзакції приблизно на 28%. Це критично важливо для масштабування додатків у реальній мережі Mainnet.